

step 1:Import required libraries

```
import nltk
import string
import pandas as pd
import matplotlib.pyplot as plt

from nltk.corpus import movie_reviews, stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
```

step 2:Download NLTK resources

```
nltk.download('movie_reviews')
nltk.download('stopwords')
```

[nltk_data] Downloading package movie_reviews to /root/nltk_data...

[nltk_data] Unzipping corpora/movie_reviews.zip.

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Unzipping corpora/stopwords.zip.

True

step 3:Land positive and negative

```
positive_reviews = [
    movie_reviews.raw(fileid)
    for fileid in movie_reviews.fileids('pos')
]

negative_reviews = [
    movie_reviews.raw(fileid)
    for fileid in movie_reviews.fileids('neg')
]

print("Positive reviews:", len(positive_reviews))
print("Negative reviews:", len(negative_reviews))
```

Positive reviews: 1000
Negative reviews: 1000

step 4:Text preproceesing function

```
stop_words = stopwords.words('english')

def preprocess(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    return text
```

step 5:Create TF-IDF vectorizer

```
vectorizer = TfidfVectorizer(
    preprocessor=preprocess,
    stop_words=stop_words,
    max_features=3000
)
```

step 6:Compute TF-IDF seperately

```
tfidf_pos = vectorizer.fit_transform(positive_reviews)
pos_features = vectorizer.get_feature_names_out()

pos_scores = tfidf_pos.mean(axis=0).A1
```

```
pos_tfidf = pd.DataFrame(
    {'term': pos_features, 'score': pos_scores}
).sort_values(by='score', ascending=False)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/feature_extraction/text.py:402: UserWarning: Your stop_words may be inconsistent
warnings.warn(
```

step 7:Top 15 TF-IDF terms

```
top_pos = pos_tfidf.head(15)

# Calculate TF-IDF for negative reviews using the same vectorizer
tfidf_neg = vectorizer.transform(negative_reviews)
neg_features = vectorizer.get_feature_names_out()

neg_scores = tfidf_neg.mean(axis=0).A1
neg_tfidf = pd.DataFrame(
    {'term': neg_features, 'score': neg_scores}
).sort_values(by='score', ascending=False)

top_neg = neg_tfidf.head(15)

print("Top 15 Positive Review Terms:")
print(top_pos)

print("\nTop 15 Negative Review Terms:")
print(top_neg)
```

Top 15 Positive Review Terms:

	term	score
1020	film	0.068548
1769	movie	0.044444
1880	one	0.042081
1568	like	0.029828
2526	story	0.024742
1170	good	0.023928
1025	films	0.023597
105	also	0.022996
1563	life	0.022874
2700	time	0.022556
444	characters	0.022319
900	even	0.021970
442	character	0.021076
2907	well	0.021074
2783	two	0.020759

Top 15 Negative Review Terms:

	term	score
1020	film	0.067441
1769	movie	0.062800
1880	one	0.045002
1568	like	0.037609
224	bad	0.036835
900	even	0.030832
2005	plot	0.027183
1170	good	0.026676
2975	would	0.025957
1143	get	0.025782
2700	time	0.024767
1775	much	0.023838
444	characters	0.023306
2526	story	0.022852
44	action	0.022527

step 8:Visualization

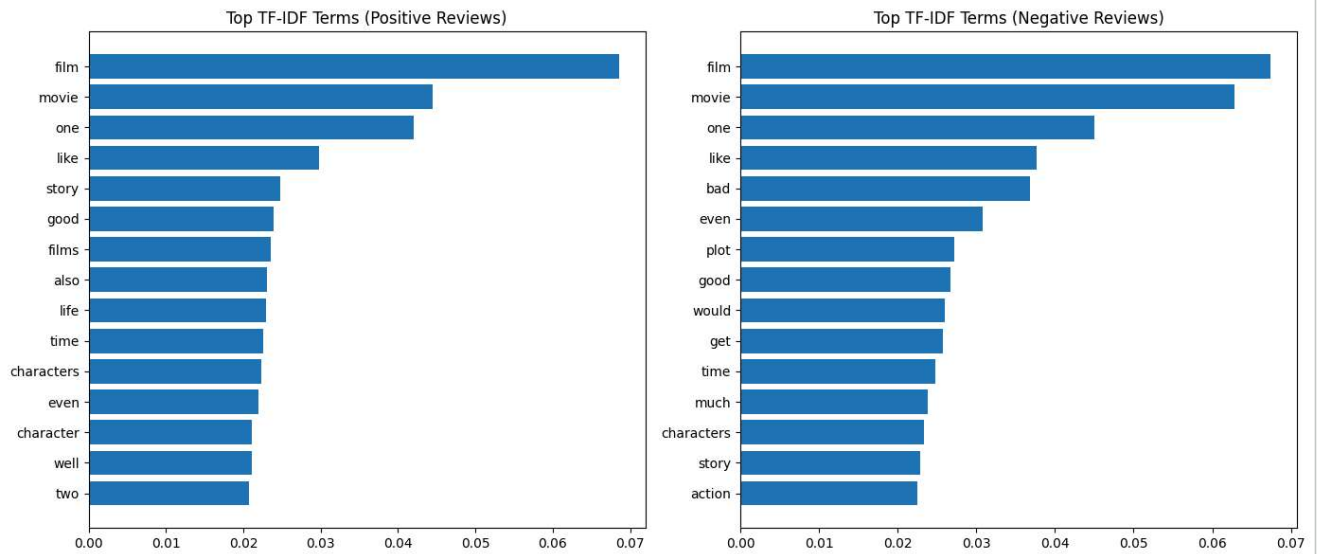
```
plt.figure(figsize=(14,6))

plt.subplot(1,2,1)
plt.barh(top_pos['term'], top_pos['score'])
plt.title("Top TF-IDF Terms (Positive Reviews)")
plt.gca().invert_yaxis()

plt.subplot(1,2,2)
plt.barh(top_neg['term'], top_neg['score'])
plt.title("Top TF-IDF Terms (Negative Reviews)")
```

```
plt.gca().invert_yaxis()
```

```
plt.tight_layout()  
plt.show()
```



Double-click (or enter) to edit